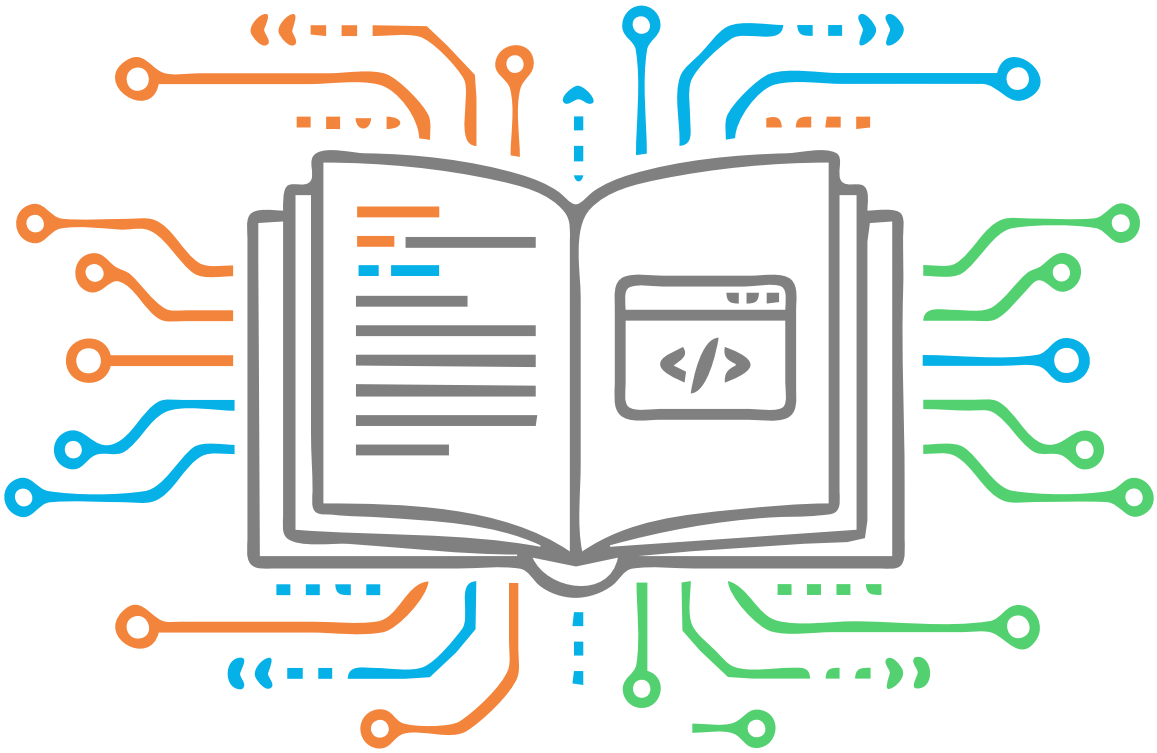


Informatik 10: Datenbanken

aktualisiert: 7. Okt. 2026

1 Wdh: Klassen und Objekte

Objekte repräsentieren **Gegenstände** in einem Computerprogramm. **Klassen** sind der **Bauplan**, der festlegt, welche **Eigenschaften** (**Attribute**) und **Fähigkeiten** (**Methoden**) einer bestimmten Objektart gespeichert werden sollen. Man stellt sie dar mit:

Klassenkarte

Person
String hobby
int alter
boolean hatHaustier
String peinlichesErlebnis
void atmen()

spitze Ecken

← Klassenname

Objektname : Klassenname →

Attribute

← Methoden

Objektkarte

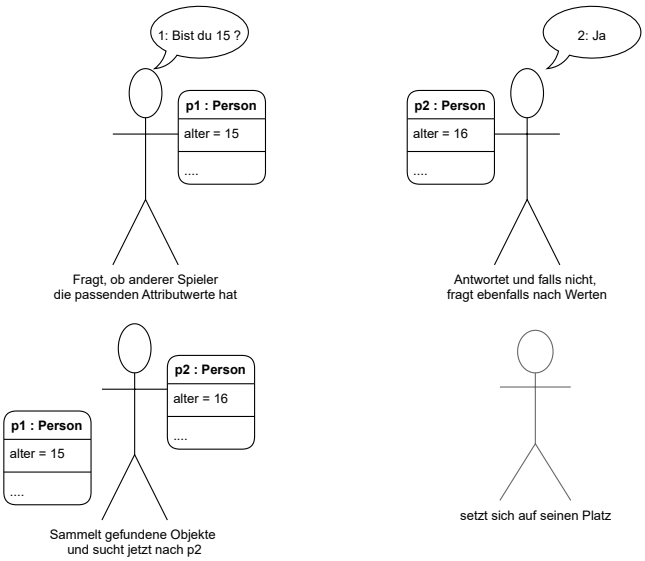
p1 : Person
hobby = "Klettern"
alter = 23
hatHaustier = false
peinlichesErlebnis = "..."

runde Ecken

ℹ Generell kann man Objektkarten mit oder ohne Methoden zeichnen, solange man es insgesamt einheitlich macht. Wir zeichnen sie daher immer ohne Methoden.

Objektkarten Memory

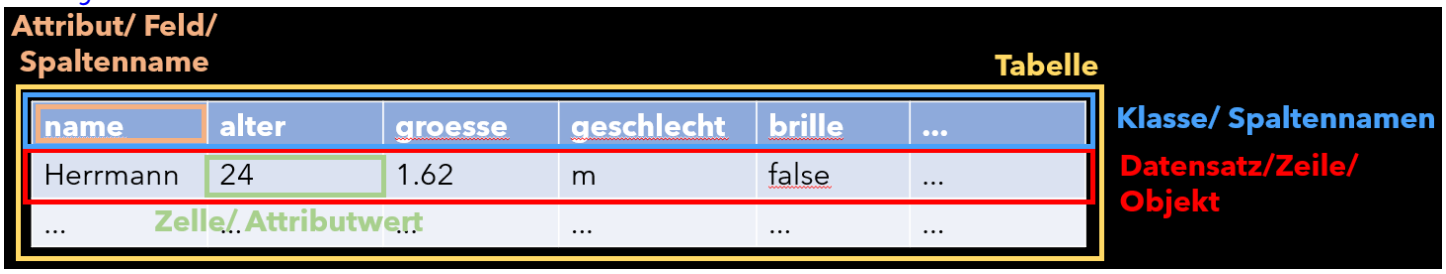
- Erstelle auf einem Blatt eine Objektkarte der Klasse Person zu dir selbst. → **3x falten**
- Gib deine Objektkarte bei der Lehrkraft ab. → Objektkarten werden gemischt.
- Ziehe eine Objektkarte und versuche, das zugehörige Objekt zu finden.
  - Frage deine:n Gegenüber dafür, ob die Attributwerte auf deiner gezogenen Karte auf sie/ihn zutreffen.
  - Ihr dürft euch nicht gegenseitig die Objektkarten zeigen!
  - Wer gefunden wurde, gibt seine aktuelle Objektkarte weiter und setzt sich.
  - Der/Die Finder:in sammelt alle gefundenen Objekte.



Wdh: Von der Klasse zur Tabelle

- Zeichnet zu zweit eine Tabelle, in der man alle Objekte der Klasse Person sammeln kann.
- Tragt eure beiden Objekte (vom Objektkarten-Memory) in die Tabelle ein.
- Ordnet die folgenden Begriffe den Teilen der Tabelle zu.
  Achtung: Nicht alle Begriffe passen und manches hat mehrere Begriffe!
  Datensatz Tabelle Zelle Klasse Objekt Parameter Attribut Spalte Feld Methode Board Zeile Datentyp Attributwert

Lösung:



Nicht verwendete Begriffe: Parameter, Methode, Board, Datentyp

Feld: Wird oft synonym zu Attribut verwendet, v.a. in Programmen wie LibreOffice Base oder MS Access.

2 Wdh: Aufbau von (relationalen) Datenbanken

Datenbanken speichern Datensätze in **Tabellen**. Die **Spaltenüberschriften** repräsentieren die **Attribute** (Synonym: Feld) und bilden zusammen eine **Klasse**. Die **Datensätze** **(=Zeilen)** entsprechen **Objekten** und in den Spalten stehen die Attributwerte. Jede Tabelle hat einen **Primärschlüssel** **(oft auch „ID“)**, der Datensätze eindeutig identifiziert. Oft werden die Datensätze hiermit einfach durchnummeriert. Im Tabellenschema wird er unterstrichen und im Klassendiagramm immer als erstes Attribut aufgelistet.

Der Aufbau einer Tabelle kann mit **Klassenkarte** oder **Tabellenschema** dargestellt werden. Dessen Aufbau ist:

TABELLENNAME(Datentyp Primärschlüssel , Datentyp Spalte1, Datentyp Spalte2, …)

Zum Beispiel:

```
Person(int id, String name, int alter, …)
```

Übung: SQL Island (1)



DORF (dorfnr, name, haeuptling)
BEWOHNER (bewohnernr, name, dorfnr, geschlecht, beruf, gold, status)
GEGENSTAND (gegenstand, besitzer)

Übung: SQL Island (2)

Oben siehst du ein reduziertes Tabellenschema von SQL Island, bearbeite folgende Aufgaben dazu:

1. Was sind die Primärschlüssel der Tabellen, die die einzelnen Objekte eindeutig identifizieren?
   → Notiert das vollständige Tabellenschema der Datenbank von SQL Island (mit Datentypen und Markierung der Primärschlüssel). Manche Datentypen sind intuitiv, andere nicht. Überlege dir, was hier sinnvoll wäre.
   ℹ Den Datentyp Character gibt es in den meisten Datenbanksystemen nicht. Wir verwenden daher immer String (=Text).
   BEWOHNER(int bewohnernr , String name, int dorfnr, String geschlecht, String beruf, int gold, String status)
   GEGENSTAND(String gegenstand, int besitzer)
   DORF(int dorfnr, String name, int haeuptling)

2. Stellt die Tabellen der Datenbank mit Klassenkarten dar.

BEWOHNER
int bewohnernr String name int dorfnr String geschlecht String beruf int gold String status

GEGENSTAND
String gegenstand int besitzer

DORF
int dorfnr String name int haeuptling

SQL Island: Zeit zu spielen

Du weißt jetzt, wie die Tabellen von SQL Island aufgebaut sind. Öffne das Spiel im Browser und nutze den SQL Spickzettel als Unterstützung!
sql-island.informatik.uni-kl.de/

3 SQL Spickzettel

Vorlage

Folgender SQL-Spickzettel enthält alle SQL-Grundlagen der 9. Klasse. Ihr dürft (sollt!) ihn bei allen SQL-Aufgaben benutzen. Über das Vorlagensymbol oben könnt ihr den Spickzettel als eigenes PDF öffnen.

ℹ SQL ist die Abkürzung für Structured Query Language, was auf Deutsch etwa Strukturierte Abfrage Sprache heißt.

Inf 9 Grundlagen	**Spickzettel SQL**	

```
SELECT    Spaltenliste   – DISTINCT vermeidet Duplikate.
                         – Aggregatfunktionen (COUNT, SUM, MAX, MIN, AVG) für Berechnungen
                         – SELECT * für "alle Spalten"
                         – AS Aliasname
FROM      Tabelle
WHERE     Bedingung      – Wird meist mit Vergleichen (<, <=, =, <>, ...) formuliert.
                         – Verknüpfung von mehreren Vergleichen mit logischen Funktionen (AND, OR, NOT)
GROUP BY  Spaltenliste
HAVING    Bedingung
ORDER BY  Spaltenliste   – ASC für aufsteigend (Standard)
                         – DESC für absteigend
```

Im Detail gilt:

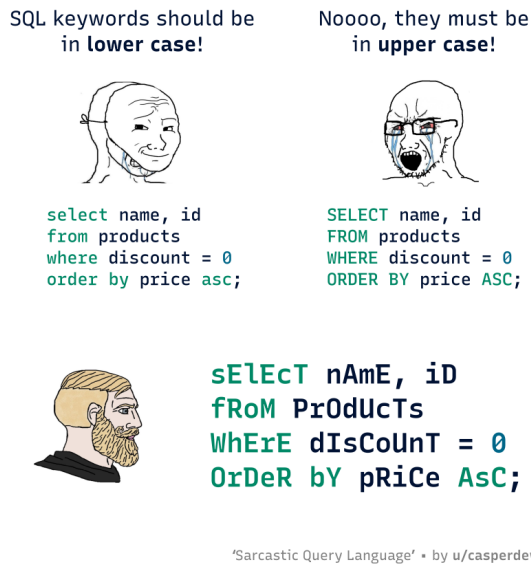
Grundlegende SQL-Abfrage	
SELECT	Es muss mindestens ein Spaltenname angegeben werden. Die entsprechende(n) Spalte(n) sind dann Teil der Ergebnistabelle. SELECT * bewirkt, dass alle Spalten angezeigt werden.
DISTINCT	Duplikate von Datensätzen werden nicht angezeigt.
AS	Eine Spalte in der Ergebnistabelle kann anders benannt werden als in der Ausgangstabelle. Dies ist vor allem bei der Verwendung von Aggregatfunktionen hilfreich.
FROM	Hier muss angegeben werden, aus welcher Tabelle die Informationen für die Abfrage genommen werden sollen.
ORDER BY	Die Ergebnistabelle wird nach der oder den angegebenen Spalten sortiert. Standardmäßig wird aufsteigend sortiert. Mit dem Zusatz DESC bzw. ASC wird absteigend bzw. aufsteigend sortiert.
Beispiele	SELECT DISTINCT kontinent AS "enthaltene Kontinente" FROM Land SELECT name, flaeche, hauptstadt FROM LAND ORDER BY flaeche DESC

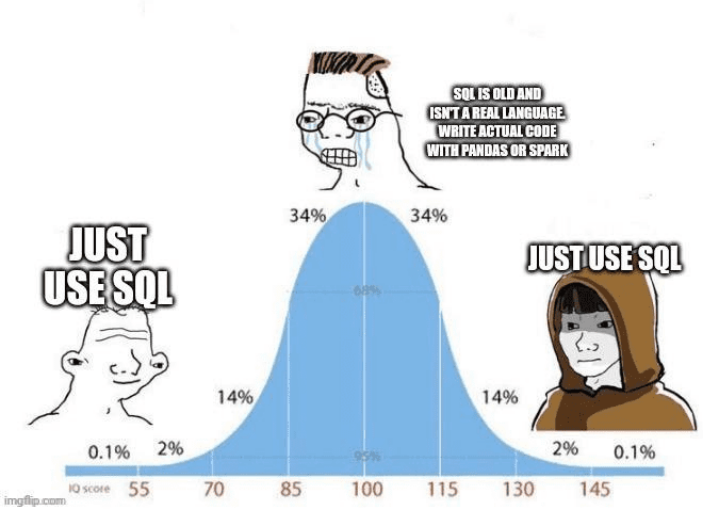
Auswahl von Datensätzen über Bedingungen	
WHERE	In der Ergebnistabelle werden nur die Datensätze (Zeilen) angezeigt, welche die angegebene Bedingung erfüllen. Eine Bedingung wird mit einem Vergleich formuliert. Neben den typischen Vergleichsoperatoren wie <, <=, =, <>, usw. sind insbesondere auch IS NULL und LIKE wichtig. Mehrere Vergleiche können durch die logischen Funktionen AND, OR und NOT verknüpft werden. Ggf. müssen die einzelnen Ausdrücke dabei sinnvoll geklammert werden Beispiel WHERE jahr > 2015 AND laufzeit <= 90 AND NOT fsk = 18
LIKE	Kann in einer Bedingung zur Mustererkennung von Einträgen verwendet werden. Folgende zwei Platzhalter (wildcards) werden häufig eingesetzt: • % steht für beliebig viele Zeichen, auch keines (* bei MS Access) • _ für genau ein beliebiges Zeichen (? bei MS Access) Beispiele: • WHERE titel LIKE "You%" – findet alle Titel die mit "You" beginnen Groß-/Kleinschreibung wird nicht berücksichtigt • WHERE titel LIKE "%love%" – findet alle Titel die "love" enthalten • WHERE titel LIKE "L___" – findet alle Titel die mit L beginnen und genau 4 Zeichen lang sind
NULL	Bedeutet, dass kein Wert in einer Zelle eingetragen ist.
IS NULL	Überprüft (in einer Bedingung), ob kein Wert in einer Zelle eingetragen ist.

Aggregatfunktionen	
AVG	Berechnet den Durchschnitt aller Werte einer Spalte.
COUNT	Gibt die Anzahl der Einträge einer Spalte aus.
MAX bzw. MIN	Gibt das Maximum bzw. Minimum aller Werte einer Spalte aus.
SUM	Berechnet die Summe aller Werte einer Spalte.
Beispiel	SELECT COUNT(*) AS "Anzahl afrikanischer Länder " FROM Land WHERE kontinent = "Afrika"

Gruppierung	
GROUP BY	Datensätze mit demselben Wert in der angegeben Spalte werden gruppiert. Gruppierungen sind nur in Kombination mit Aggregatfunktionen sinnvoll.
HAVING	An gruppierte Datensätze werden Bedingungen mit HAVING formuliert.
Beispiel	SELECT fsk, MIN(laufzeit) FROM Film WHERE genre1="Filmkomödie" OR genre2="Filmkomödie" GROUP BY fsk HAVING fsk <16

Quelle des Spickzettels: https://informatikschulbuch.de/jahrgangsstufe-9-uebersicht/datenmodellierung-und-datenbankenjgst10/





ℹ SQL Schlüsselwörter wie SELECT, WHERE etc. sind nicht case-sensitive. Groß-/Kleinschreibung ist also egal.



SQL Puzzle

In dieser Aufgabe geht es immer um die Tabelle **land**, deren erste Datensätze du hier siehst:

id	name	einwohner	flaeche	hauptstadt
1	Deutschland	83.24	358	Berlin
2	Frankreich	67.39	544	Paris
3	Brasilien	212.60	8516	Rio de Janeiro
...	...	...	...	...

Welche SQL-Abfrage (rechte Seite) führt zu welcher Ergebnistabelle (linke Seite)? Ordne richtig zu!

Lösung:

```
1) iv)      4) i)       7) v)
2) viii)    5) ix)      8) ii)
3) vii)     6) iii)     9) vi)
```

1) Zeige alle Spalten der Tabelle land.

2) Zeige die Spalten name und hauptstadt der Tabelle land.

3) Zeige die durchschnittliche Einwohnerzahl aller Länder.

4) Zeige die Namen aller Länder in alphabetisch absteigender Reihenfolge.

5) Zeige die Hauptstädte der Länder, deren Einwohnerzahl größer als 50 Mio ist.

6) Zeige die Anzahl aller Länder, deren Name mit 'land' endet.

7) Zeige die Namen aller Länder, deren Fläche zwischen 100 und 999 Tausend km² liegt.

8) Zeige die Namen der Länder, die mit 'D' beginnen oder mit 'd' aufhören.

9) Zeige die Namen der drei Länder mit der größten Einwohnerzahl.

i)
```
SELECT name
FROM land
ORDER BY name DESC;
```

ii)
```
SELECT name
FROM land
WHERE name LIKE 'D%'
OR name LIKE '%d';
```

iii)
```
SELECT COUNT(*)
FROM land
WHERE name LIKE '%land';
```

iv)
```
SELECT *
FROM land;
```

v)
```
SELECT name
FROM land
WHERE flaeche >= 100
AND flaeche <= 999;
```

vi)
```
SELECT name, einwohner
FROM land
ORDER BY einwohner DESC
LIMIT 3;
```

vii)
```
SELECT AVG(einwohner)
FROM land;
```

viii)
```
SELECT name, hauptstadt
FROM land;
```

ix)
```
SELECT hauptstadt
FROM land
WHERE einwohner > 50;
```



Wdh: SQL Basics

Bearbeite die Aufgabe **Wdh - SQL Basics** auf `artemis.tum.de`. Artemis gibt dir immer, wenn du auf Submit drückst, die ersten Zeilen der Ergebnistabelle und ob deine SQL-Abfrage (bzw. welche Teile von ihr) richtig sind, aus.
Wenn du eine Abfrage richtig hast, notiere sie unten im Skript.
Falls du bei Gruppierung und Aggregatfunktionen Schwierigkeiten hast, hilft dir dieses **Video (bitte Kopfhörer verwenden!)**: `bycs.link/simpleclub-group-sort-aggregat`

1) Vervollständige die SQL-Abfrage so, dass sie ID, Name, Art und URL aller Freibäder ausgibt.

```
 SELECT id, name, art, url
FROM Schwimmbad
WHERE art="Freibad"
```

Wdh: SQL Basics

2) Schreibe eine SQL-Abfrage, die ausgibt, wie viele Gemeinden es im Regierungsbezirk "Oberbayern" gibt.

```
SELECT COUNT(*)
FROM Gemeinde
WHERE regierungsbezirk="Oberbayern"
```

3) Schreibe eine SQL-Abfrage, die Name, Straße und URL (also die Internetadresse) alle Zoos in der Gemeinde mit Schluessel "09162000" ausgibt.

```
SELECT name, strasse, url
FROM Zoo
WHERE gemeindeschluessel = "09162000"
```

Wdh: SQL Basics

4) Schreibe eine SQL-Abfrage, die die Summe aller weiblichen Einwohnerinnen und die Summe aller männlichen Einwohner gruppiert nach Regierungsbezirk und den Namen des jeweiligen Regierungsbezirks ausgibt.

```
SELECT regierungsbezirk, SUM(einwohner_w), SUM(einwohner_m)
FROM gemeinde
GROUP BY regierungsbezirk
```

5) Schreibe eine SQL-Abfrage, die die durchschnittliche Fläche der Gemeinde eines Kreises (=Landkreis) und den Namen und Regierungsbezirk des jeweiligen Landkreises anzeigt. Sortiere die Ausgabe nach Name des Landkreises.
ℹ Achtung: Du kannst bei der Verwendung von Gruppierung nur Spalten, nach denen gruppiert wird, und solche, die mit Aggregatfunktionen zusammengefasst werden, anzeigen! Überlege, wie du dieses Problem hier lösen kannst.

```
SELECT regierungsbezirk, kreis, avg(flaeche)
FROM Gemeinde
GROUP BY regierungsbezirk,kreis
ORDER BY kreis
```

Wdh: SQL Basics

6) Schreibe eine SQL-Abfrage, die die Namen und Einwohnerzahlen aller Gemeinde, die mehr als 100.000 männliche und mehr als 100.000 weibliche Einwohner:innen haben, ausgibt.

```
SELECT name, einwohner_m, einwohner_w
FROM Gemeinde
WHERE einwohner_m > 100000
AND einwohner_w > 100000
```

7) Schreibe eine SQL-Abfrage, die die Namen und Einwohnerzahlen aller Gemeinde, die mehr als 75.000 männliche oder mehr als 75.000 weibliche Einwohner:innen haben, ausgibt.

```
SELECT name, einwohner_m, einwohner_w
FROM Gemeinde
WHERE einwohner_m > 75000
OR einwohner_w > 75000
```

Wdh: SQL Basics

8) Schreibe eine SQL-Abfrage, die Name, Landkreis, Fläche und die Einwohnerzahlen aller Gemeinden ausgibt, die jeweils mehr als 50.000 männliche und weibliche Einwohner:innen oder eine Fläche größer als 100 km² hat.

```
SELECT name, kreis, flaeche, einwohner_m, einwohner_w
FROM Gemeinde
WHERE (einwohner_m > 50000 AND einwohner_w > 50000)
OR flaeche > 100
```

9) Schreibe eine SQL-Abfrage, die die durchschnittlichen männlichen und weiblichen Einwohnerzahlen aller Gemeinde mit mehr als 100 km² Fläche pro Landkreis und den Namen des jeweiligen Landkreises ausgibt.

```
SELECT kreis, AVG(einwohner_m), AVG(einwohner_w)
FROM Gemeinde
WHERE flaeche > 100
GROUP BY kreis
```

Wdh: SQL Basics

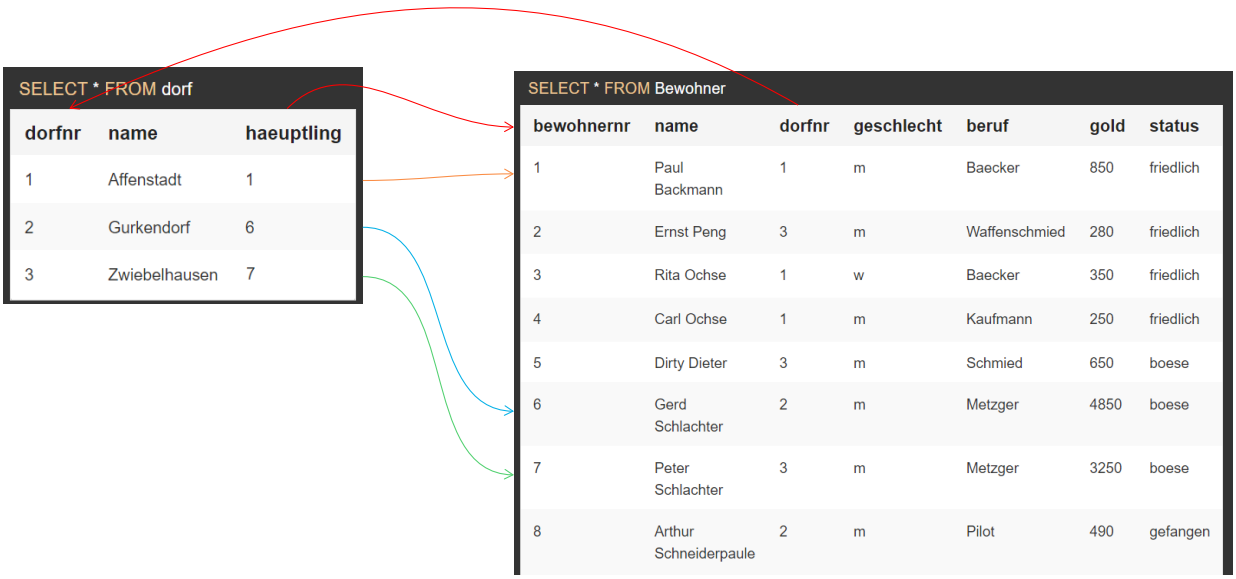
10) Schreibe eine SQL-Abfrage, die die Anzahl von Wanderwegen, die zu einer Gemeinde führen in einer Spalte Anzahl und den jeweiligen Gemeindeschlüssel absteigend nach Anzahl sortiert, ausgibt.

```
SELECT gemeindeschluessel,COUNT(*) as Anzahl
FROM Wanderweg_zu_Gemeinde
GROUP BY gemeindeschluessel
ORDER BY Anzahl DESC
```



Tabellenbeziehungen

1. Visualisiere (mit Bleistift), wer Häuptling in welchem Dorf ist.
2. Überlege, wie du allgemein für diese zwei Tabellen darstellen kannst, wie sie (und ihre Spalten) miteinander in Beziehung stehen.



SELECT * FROM dorf

dorfnr	name	haeuptling
1	Affenstadt	1
2	Gurkendorf	6
3	Zwiebelhausen	7

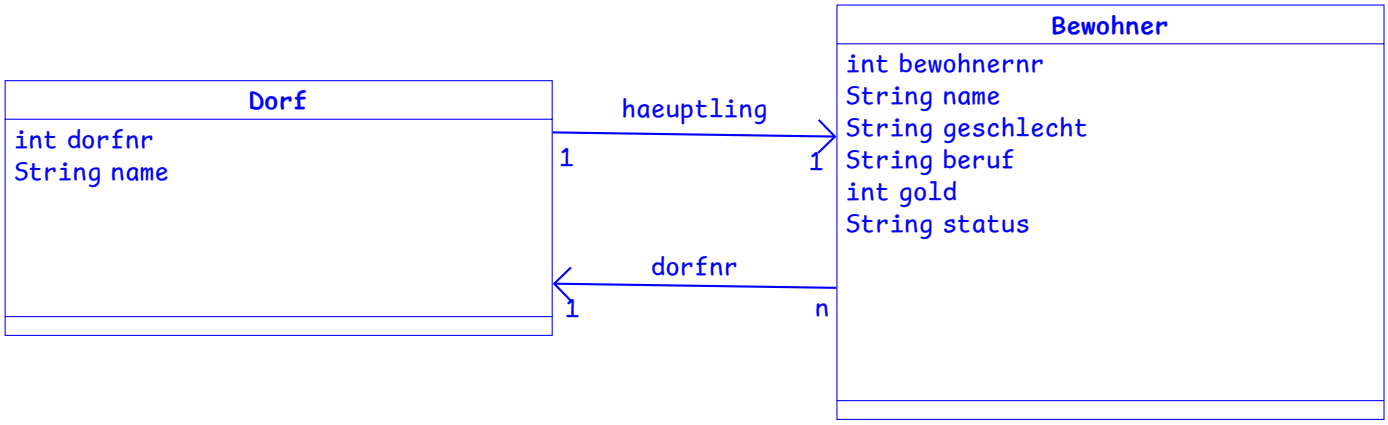
SELECT * FROM Bewohner

bewohnernr	name	dorfnr	geschlecht	beruf	gold	status
1	Paul Backmann	1	m	Baecker	850	friedlich
2	Ernst Peng	3	m	Waffenschmied	280	friedlich
3	Rita Ochse	1	w	Baecker	350	friedlich
4	Carl Ochse	1	m	Kaufmann	250	friedlich
5	Dirty Dieter	3	m	Schmied	650	boese
6	Gerd Schlachter	2	m	Metzger	4850	boese
7	Peter Schlachter	3	m	Metzger	3250	boese
8	Arthur Schneiderpaule	2	m	Pilot	490	gefangen

Tabellenbeziehung im Klassendiagramm

Tabellenbeziehung im Klassendiagramm

1. Ergänze das Klassendiagramm entsprechend den beiden Tabellen oben.
2. Wie kann man die Beziehungen zwischen den beiden Tabellen im Klassendiagramm darstellen?
   Tipp: Unsere Überlegungen von oben helfen dabei.



4 Tabellenbeziehungen: Fremdschlüssel

Wenn Datensätze mittels Primärschlüssel in einer anderen Tabelle verwendet werden, spricht man dort von einem Fremdschlüssel. Im Tabellenschema werden die **Fremdschlüssel** durch überstreichen (manchmal auch unterpunkten) markiert. Ein Beispiel in SQL-Island ist der Häuptling eines Dorfes, der in der Tabelle Dorf mittels bewohnernr eingetragen wird. Die bewohnernr ist hierbei **Primärschlüssel** in der Tabelle Bewohner und **Fremdschlüssel** in der Tabelle Dorf (heißt hier aber haeuptling).

5 Tabellenbeziehungen im Klassendiagramm



- Beziehungspfeil immer vom Fremd- zum Primärschlüssel.
- 'fremdschluessel' ist eine Spalte der TabelleA, wird dort aber nicht eingetragen.
- Die Form der Pfeilspitze ist wichtig und muss genau so sein, da andere Spitzen andere Bedeutungen haben!
- Kardinalität an der Pfeilspitze ist immer 1 (bei Datenbanken), da in einer Spalte (eines Datensatzes) immer nur ein Wert stehen kann.

6 Kardinalitäten

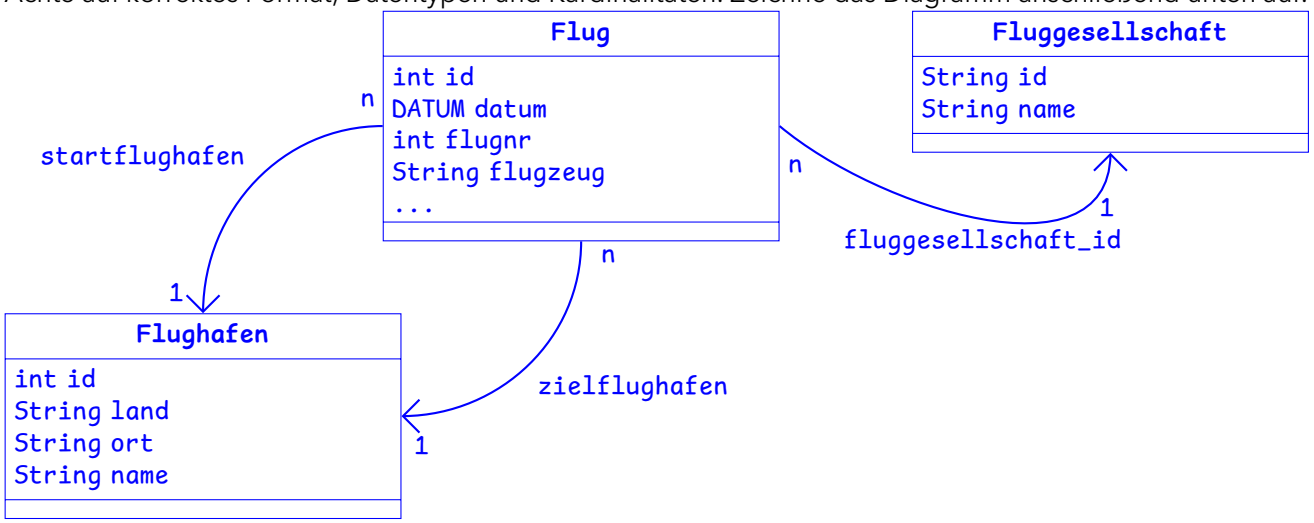
Die Kardinalität beschreibt, wie viele Objekte auf jeder Seite einer Beziehung stehen können.

Es gibt folgende Arten:

- 1:1, z.B. `ein` Häuptling pro Dorf, der auch nur in einem Dorf Häuptling ist.
- 1:n, z.B. jeder Bewohner wohnt in einem Dorf, das aber `mehrere` Bewohner hat.
- m:n, z.B. `beliebig viele` Lehrer pro Schulklasse + `beliebig viele` Schulklassen pro Lehrer (in Datenbanken nicht direkt umsetzbar, dazu später mehr).

Klassendiagramm Flugverspätung

Bearbeite diese Aufgabe auf `artemis.tum.de`.
Erstelle ein Klassendiagramm für die Datenbank unter `dbiu.de/flugverspaetungen/` .
Damit du weniger schreiben musst, kannst du die **letzten 6** Spalten der **Tabelle Flug** durch **... ersetzen**.
Achte auf korrektes Format, Datentypen und Kardinalitäten. Zeichne das Diagramm anschließend unten auf:



SQL: Tabellen verbinden

Wir kennen jetzt Tabellen, die miteinander über Fremd- und Primärschlüssel in Beziehung stehen. Nun möchten wir aus diesen Tabellen auch zusammengehörende Datensätze abfragen.
Öffne dafür `www.dbiu.de/studierende/` und führe folgende SQL-Abfrage aus (Groß-Kleinschreibung beachten!):

```
SELECT *
FROM Studierende, Studiengang
```

SQL: Tabellen verbinden

Was beobachtest du? Werden nur zusammengehörende Datensätze angezeigt? Falls nicht, nach welchem Muster werden die beiden Tabellen miteinander kombiniert?
Nein, es werden alle Datensätze aus einer mit allen Datensätzen aus der anderen kombiniert und die Spalten einfach hintereinander aufgereiht.

7 Kreuzprodukt / Join

Möchte man Daten aus zwei Tabellen mit Beziehung zueinander abfragen, gibt man beide Tabellen **mit Komma getrennt nach FROM** an.

Die SQL-Abfrage bildet dann das **Kreuzprodukt** der Tabellen. Die Ergebnistabelle enthält **alle Kombinationen** von Datensätzen beider Tabellen **(Merkregel: Jeder mit Jedem)**.

Um nur zusammengehörige Datensätze (also solche, die miteinander in Beziehung stehen, z.B. ein Bewohner mit seinem Dorf) auszuwählen, ergänzt man als **Selektion** eine **Gleichheitsbedingung** zwischen Fremd- und zugehörigem **Primärschlüssel** . Dann spricht man von einem **Join** .

Zum Beispiel kann man in SQL-Island die Daten aller Dörfer und ihrer zugehörigen Häuptlinge so ausgeben:

```
SELECT *
FROM Dorf, Bewohner
WHERE Dorf.haeuptling = Bewohner.bewohnernr
```

8 Join Beispiel

Lehrkraft

id	kuerzel	schule
1	Her	MTG
2	Gra	Dante

```
SELECT *
FROM Lehrkraft, Schule
WHERE Lehrkraft.schule = Schule.id
```

Schule

id	ort
MTG	Haidh.
Dante	Sendl.

Ergebnistabelle des Kreuzprodukts:

id	kuerzel	schule	id	ort
1	Her	MTG	MTG	Haidh.
2	Gra	Dante	MTG	Haidh.
1	Her	MTG	Dante	Sendl.
2	Gra	Dante	Dante	Sendl.

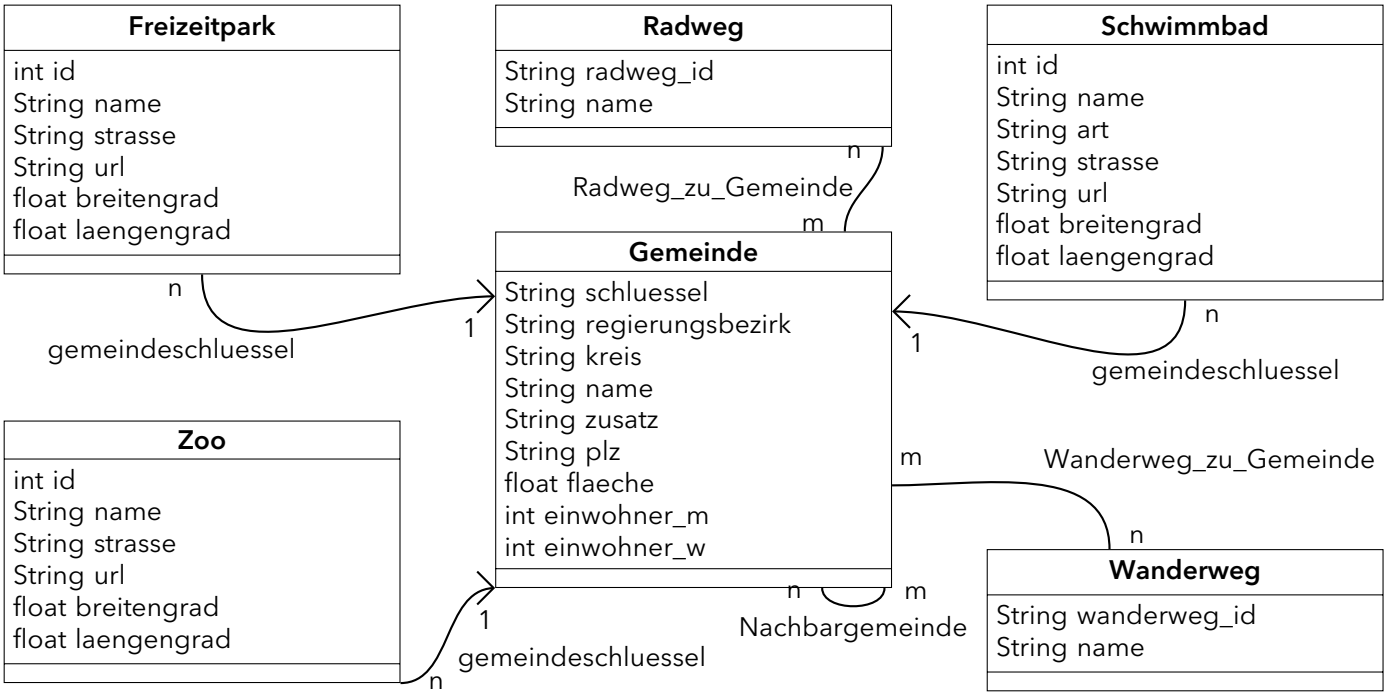
Ergebnistabelle des Joins

id	kuerzel	schule	id	ort
1	Her	MTG	MTG	Haidh.
2	Gra	Dante	Dante	Sendl.

SQL mit Kreuzprodukt und Join

Bearbeite diese Aufgabe auf artemis.tum.de. Du bekommst eine automatische Rückmeldung, ob deine Abgabe korrekt ist. Alle Aufgaben beziehen sich auf die Datenbank mit unten stehendem Klassendiagramm. Eine Online-Version gibt es unter www.dbiu.de/bayern/, dort ist auch das Tabellenschema zu finden.
Gib immer genau die geforderten Daten aus und nicht mehr. Sortiere nicht, wenn du nicht dazu aufgefordert wirst.
Notiere anschließend deine korrekten SQL-Abfragen unten.

SQL mit Kreuzprodukt und Join



SQL mit Kreuzprodukt und Join

Verändere die SQL-Abfrage so, dass die Namen und Internetadressen (=url) aller Zoos und der Name und Regierungsbezirk der jeweiligen Gemeinde ausgegeben werden:

```
SELECT Zoo.name, Gemeinde.name    ,Gemeinde.regierungsbezirk, Zoo.url

FROM Zoo, Gemeinde

WHERE Zoo.gemeindeschluessel = Gemeinde.schluessel
```

SQL mit Kreuzprodukt und Join

Verändere die SQL-Abfrage so, dass die Namen und Straßen aller Freizeitparks und die Namen der jeweils zugehörigen Gemeinde ausgegeben werden.

```
SELECT Freizeitpark.name, Gemeinde.name    , Freizeitpark.strasse

FROM Freizeitpark, Gemeinde

WHERE Gemeinde.schluessel = Freizeitpark.gemeindeschluessel
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die Namen und Art aller Schwimmbäder und den Namen und alle Einwohnerzahlen der zugehörigen Gemeinden ausgibt.

```
SELECT Schwimmbad.name, Schwimmbad.art,
Gemeinde.name, Gemeinde.einwohner_m, Gemeinde.einwohner_w
FROM Schwimmbad, Gemeinde
WHERE Gemeinde.schluessel = Schwimmbad.gemeindeschluessel
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die die Anzahl an Schwimmbädern in Gemeinden mit **mehr** als 1000 weiblichen Einwohnerinnen ausgibt.

Tipp: Hier brauchst du mehrere verknüpfte Bedingungen

```
SELECT COUNT(*)
FROM Schwimmbad, Gemeinde
WHERE Gemeinde.schluessel = Schwimmbad.gemeindeschluessel
AND Gemeinde.einwohner_w > 1000
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die die Namen aller Gemeinde in Oberbayern oder Niederbayern, zu denen ein Wanderweg führt, ausgibt. Dopplungen dürfen auftreten und sollte nicht entfernt werden!

Tipp: Hier brauchst du wieder mehrere verknüpfte Bedingungen. Überlege bei der Verknüpfung von Bedingungen, ob du Klammern setzen musst!

```
SELECT Gemeinde.name
FROM Gemeinde,Wanderweg_zu_Gemeinde
WHERE Gemeinde.schluessel = Wanderweg_zu_Gemeinde.gemeindeschluessel
AND (Gemeinde.regierungsbezirk='Oberbayern'
OR Gemeinde.regierungsbezirk='Niederbayern')
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die aus den Tabellen Gemeinde und Wanderweg_zu_Gemeinde die Anzahl der Wanderwege, die zu Gemeinden mit mehr als 500 000 männlichen Einwohnern führen, ausgibt.

```
SELECT COUNT(*)
FROM Gemeinde, Wanderweg_zu_Gemeinde
WHERE Gemeinde.schluessel = Wanderweg_zu_Gemeinde.gemeindeschluessel
AND einwohner_m > 500000
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die eine Liste mit den Namen aller Gemeinden, die ein 'Freibad' haben, und die Namen der jeweiligen Freibäder ausgibt.

```
SELECT Gemeinde.name, Schwimmbad.name
FROM Gemeinde, Schwimmbad
WHERE Gemeinde.schluessel=Schwimmbad.gemeindeschluessel
AND Schwimmbad.art='Freibad'
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die die Anzahl an Radwegen, die an Gemeinden im PLZ-Bereich größer als 96400 angrenzen, ausgibt.

```
SELECT COUNT(*)
FROM Gemeinde, Radweg_zu_Gemeinde
WHERE Gemeinde.schluessel=Radweg_zu_Gemeinde.gemeindeschluessel
AND Gemeinde.plz > 96400
```



SQL mit Kreuzprodukt und Join

Schreibe eine SQL-Abfrage, die die Namen aller Zoos in einer Gemeinde namens 'Erlangen' ausgibt.

```
SELECT Zoo.name
FROM Zoo,Gemeinde
WHERE Zoo.gemeindeschluessel = Gemeinde.schluessel
AND Gemeinde.name='Erlangen'
```



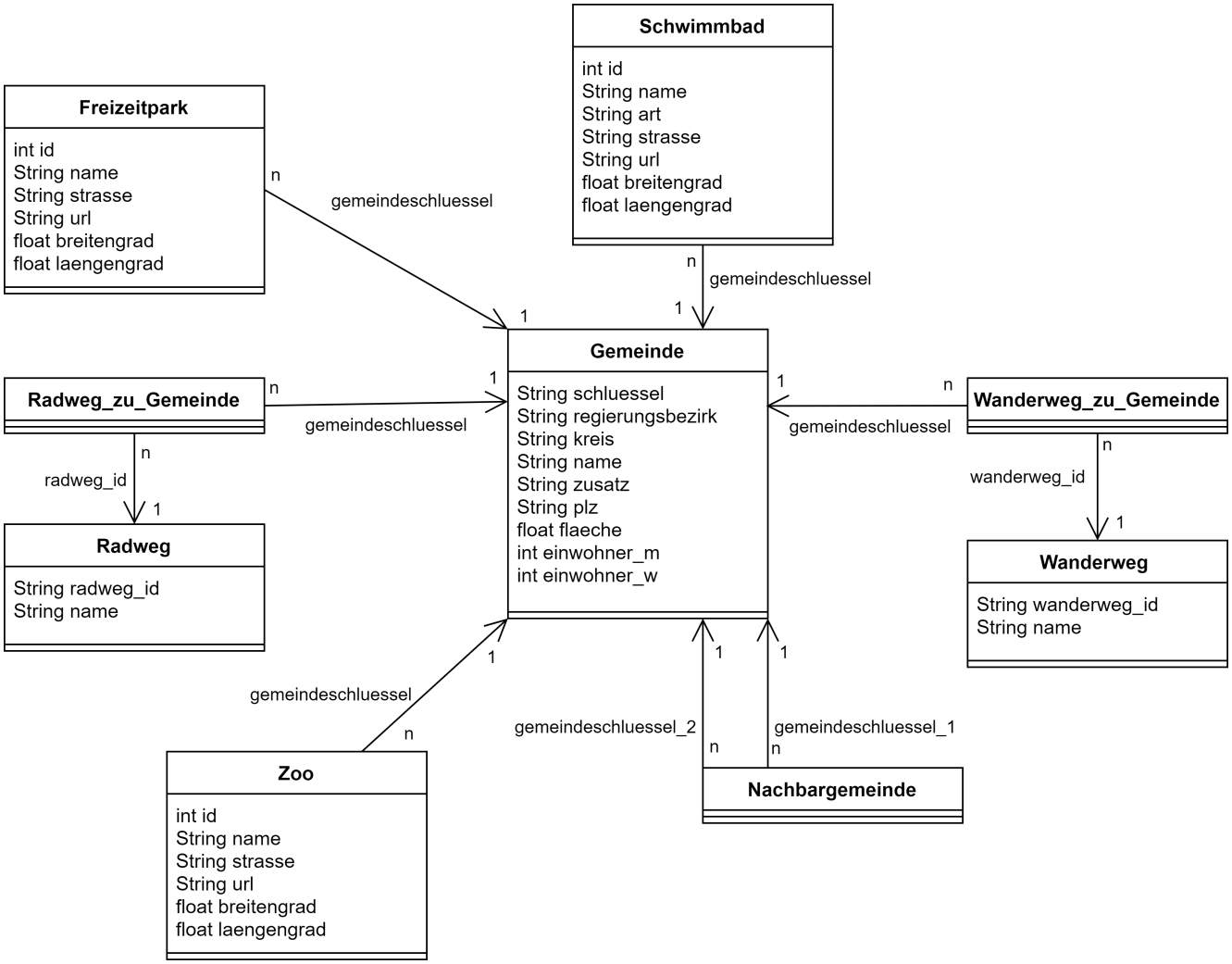
SQL mit Kreuzprodukt und Join

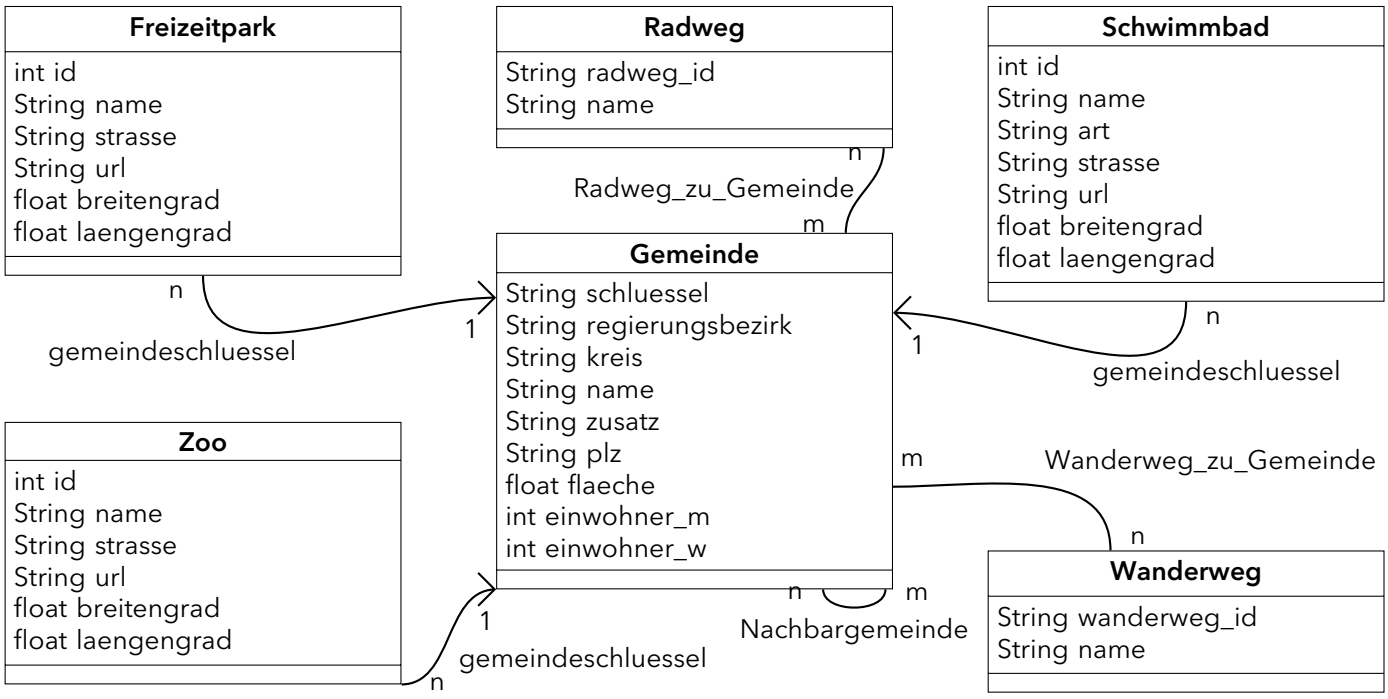
Schreibe eine SQL-Abfrage, die die IDs aller Radwege, die zu Gemeinden in Oberfranken oder Unterfranken führen, ausgibt. Dopplungen sollen nicht entfernt werden.

```
SELECT Radweg_zu_Gemeinde.radweg_id
FROM Radweg_zu_Gemeinde, Gemeinde
WHERE Gemeinde.schluessel = Radweg_zu_Gemeinde.gemeindeschluessel
AND (Gemeinde.regierungsbezirk = 'Oberfranken'
OR Gemeinde.regierungsbezirk='Unterfranken')
```

Zwei Diagramme für eine Datenbank?

Folgende Diagramme stellen dieselbe Datenbank dar. Finde Unterschiede und erkläre wieso sie auftreten. Nutze bei Bedarf auch `www.dbiu.de/bayern/`







m:n-Beziehungen

1. Erstelle in `www.sqlution.de` ein Datenbankmodell mit den Klassen Lehrkraft und Schulklasse, die in einer m:n-Beziehung zueinander stehen. Überlege dir sinnvolle Primärschlüssel und 1-2 Attribute und eine sinnvolle Bezeichnung für die Beziehung.
2. Generiere nun die zugehörige Datenbank und befülle die Tabellen mit jeweils 2-3 Datensätzen (mittels SQL).
   → Für das Einfügen von Datensätzen brauchst du diesen Befehl
   INSERT INTO <tabelle> VALUES (<wertSpalte1>, <wertSpalte2>, ...)
   mehr Infos dazu findest du hier: `https://www.w3schools.com/sql/sql_insert.asp`
3. Beantworte folgende Fragen:
   - Welchen essentiellen Unterschied gibt es zwischen m:n-Beziehungen und 1:n-/1:1-Beziehungen?
     `Beziehungstabelle benötigt`
   - Was für Datensätze werden in der dritten Tabelle eingetragen?
     `Paare von IDs der Datensätze der anderen Tabellen, die in Beziehung zueinander stehen`
   - Welche Spalten welcher Tabelle(n) sind Fremdschlüssel?
     `beide Spalten der dritten Beziehungstabelle`
   - Welche Spalte(n) sind Primärschlüssel in der dritten Tabelle? **Tipp: Was muss eindeutig sein? Probiere deine Vermutung aus, indem du versucht mehrere Datensätze mit gleichem (vermuteten) Primärschlüssel einzufügen.**
     `beide Spalten zusammen`
   - Ist es sinnvoll bei m:n-Beziehungen im Klassendiagramm eine Richtung anzugeben und wieso?
   - Wie könnte man m:n-Beziehung im Klassendiagramm alternativ darstellen?
     `zusätzliche Beziehungstabelle mit zwei 1:n-Beziehungen`

9 Darstellung von m:n-Beziehungen

m:n-Beziehungen können im UML-Klassendiagramm auf zwei verschiedene Arten dargestellt werden:

1. **als direkte Beziehung**

Vorteil: Diagramm kompakt und übersichtlich

2. **mit Beziehungstabelle**

Vorteil: Diagramm kompakt und übersichtlich

Song-Datenbank Diagramme: www.dbiu.de/songs

Zeichne die **Klassenkarten** der Tabellen und **Song und Playlist**. Zeichne anschließend mit **zwei verschiedenen Farben** die beiden **Darstellungsmöglichkeiten der Beziehung** zwischen den beiden Tabellen ein.

10 m:n Beispiel



Lehrer

id	kuerzel
1	Her
2	Gra

lehrer_an_schule

lehrer	schule
1	MTG
2	Dante
2	MTG

Schule

id	ort
MTG	Haidh.
Dante	Sendl.

Ergebnistabelle des Kreuzprodukts:

id	kuerzel	lehrer	schule	id	ort
1	Her	1	MTG	MTG	Haidh.
1	Her	1	MTG	Dante	Sendl.
1	Her	2	Dante	MTG	Haidh.
1	Her	2	Dante	Dante	Sendl.
1	Her	2	MTG	MTG	Haidh.
1	Her	2	MTG	Dante	Sendl.
2	Gra	1	MTG	MTG	Haidh.
2	Gra	1	MTG	Dante	Sendl.
2	Gra	2	Dante	MTG	Haidh.
2	Gra	2	Dante	Dante	Sendl.
2	Gra	2	MTG	MTG	Haidh.
2	Gra	2	MTG	Dante	Sendl.

insgesamt 2 x 3 x 2 = 12 Zeilen

2 Spaltenpaare müssen zusammenpassen! ⇒ Doppel-Join

```
SELECT *
FROM Lehrer, lehrer_an_schule, Schule
WHERE Lehrer.id = lehrer_an_schule.lehrer
AND lehrer_an_schule.schule = Schule.id
```

Ergebnistabelle des Doppel-Joins:

id	kuerzel	lehrer	schule	id	ort
1	Her	1	MTG	MTG	Haidh.
2	Gra	2	Dante	Dante	Sendl.
2	Gra	2	MTG	MTG	Haidh.

nur noch 3 Zeilen – genau die Zuordnungen aus lehrer_an_schule

11 SQL-Abfragen mit Join bei m:n-Beziehungen



Um zwei Tabellen, die eine m:n-Beziehung miteinander haben, zu joinen (also ihren Join zu bilden und in der Ergebnistabelle nur **zusammengehörende** Datensätze zu haben), muss man:

- Daten aus allen **drei** Tabellen abfragen (also diese nach **FROM** auflisten).
- Die **Beziehungstabelle** einzeln mit den normalen Tabellen joinen. Hierfür benötigt man **zwei** Join-Bedingungen, die mit **AND** verknüpft werden.

Beispiel:

```
SELECT Lehrkraft.*, Schulklasse.*
FROM Lehrkraft, Schulklasse, Lehrer_unterricht_Klasse
WHERE Lehrer_unterricht_Klasse.lehrer = Lehrkraft.kuerzel
AND Lehrer_unterricht_Klasse.klasse = Schulklasse.bezeichner
```



Song-Datenbank SQL

Bearbeite dann folgende SQL-Aufgaben auf der Website `www.dbiu.de/songs` und notiere die getesteten Abfragen.

1. Welche Songs (alle Attribute) sind in irgendeiner Playlist enthalten?
   ```
   SELECT Song.* FROM Song, Song_in_Playlist WHERE Song_in_Playlist.song_id = Song.id
   ```

2. Gib die Titel aller Playlists und die Titel der jeweils zugehörigen Songs aus.
   ```
   SELECT Playlist.titel, Song.titel FROM Song, Playlist, Song_in_Playlist WHERE Song_in_Playlist.song_id = Song.id AND Song_in_Playlist.playlist_id = Playlist.id
   ```

3. Welche Songs (alle Attribute) sind in der Playlist namens 'Fussballhits' enthalten?
   ```
   SELECT Song.* FROM Song, Song_in_Playlist, Playlist WHERE Song_in_Playlist.song_id = Song.id AND Song_in_Playlist.playlist_id = Playlist.id AND Playlist.titel = 'Fussballhits'
   ```